Lab 2 Part 3 報告：Calibration-Free Quantization

學號：315605019　　日期：2026-11-08

Method

我重現的是 AdaptQuant，方法本身不再介紹，以下說明實作的細節、為了符合 5.3 bits 上限所做的修改，以及做法的選擇。

選擇 AdaptQuant 的理由

這部分要求量化 Qwen3-4B 時不能用校正資料。AdaptQuant 只看每個 tile 自己的統計量來決定格式，門檻是在別的模型上事先算好的常數，量化目標模型時完全不需要資料，符合要求。它也是作業指定的比較基準，論文有 Qwen3-4B 的數字（PPL 13.98）可以直接對照。

程式架構

程式分成三個檔案：`mx.py` 是 MX 和 NVFP4 的量化器、格式選擇與 bits 計算；`derive.py` 在 Llama-3.1-8B 上推導門檻，只需要執行一次；`run_ptq.py` 讀取門檻、量化 Qwen3-4B、統計 bits，再用課程提供、未修改的 `eval_ppl_wikitext2` 評估。

T4 不支援 MX 格式，所以全部用 fake quantization：量化後馬上反量化回 fp16。量化範圍是 36 層、每層 7 個 Linear 的權重和輸入 activation；lm_head 和 attention 裡的 QK^T、PV 維持 fp16，論文也只處理 Linear 的矩陣乘法。報告中的程式片段為了好讀有稍微簡化，完整版本見程式檔。

1. MX 量化器

```
def mx_quantize(x, fmt):
    xb = x.float().reshape(*shape[:-1], shape[-1] // 32, 32)   # 32 個一個 block
    amax = xb.abs().amax(dim=-1, keepdim=True)
    emax = math.floor(math.log2(ELEM[fmt][2]))                 # 元素格式最大值的指數
    sigma = torch.floor(torch.log2(amax)) - emax
    s = torch.where(amax > 0, torch.exp2(sigma), torch.ones_like(amax))  # 2 的冪次
    return (fp_round(xb / s, fmt) * s).reshape(shape)
```

這照 OCP MX 規格的轉換方式：scale 取「block 最大值的 2 的冪次」除以「元素格式能表示的最大 2 的冪次」，所以 scale 一定是 2 的冪次（E8M0）。`fp_round()` 在元素格點上做 round-to-nearest-even，超過最大值就 clamp。三種元素格式是 E2M1（FP4，最大 6）、E2M3（FP6，最大 7.5）、E4M3（FP8，最大 448）。

2. 每個 tile 選格式

```
def tile_stat(x):                        # x: [R, K]，K 是相乘的維度
    t = to_tiles(x.float())              # -> [R/16, K/32, 16, 32]
    amax = t.abs().amax(dim=(-1, -2))
    rms = t.pow(2).mean(dim=(-1, -2)).sqrt()
    return torch.log2(amax / rms)

def adaptquant(x, t1, t2):
    # 0 = FP4, 1 = FP6, 2 = FP8
    fmt = (tile_stat(x) >= t1).long() + (tile_stat(x) >= t2).long()
    out = mx_quantize(x, "fp4")
    q = mx_quantize(x, "fp6")
    # FP6 的 tile 換成 FP6 的結果（FP8 同理）
    out = torch.where(mask_of(fmt == 1), q, out)
    return out, fmt
```

每個 16 × 32 的 tile 算一個 stat = log2(amax / rms)，代表這個 tile 裡「典型的值比最大值小多少」。stat 越大，小的值越容易在 FP4 下被捨成 0，就改用 FP6。權重的 tile 取 16 個輸出 row × 32 個輸入 column，activation 取 16 個 token × 32 個 channel，32 都落在相乘的維度上，和 MX 的 block 方向一致。

3. 門檻推導與 bits 上限

`derive.py` 照論文的 Algorithm 1，在 Llama-3.1-8B 上跑一次。每一層、每種 projection 的 tile 依 stat 分到 0 到 4.5、間隔 0.02 的格子裡，記下每格的 tile 數，以及這些 tile 用 FP4、FP6、FP8、NVFP4 量化時、以「對輸出的影響」加權的誤差。一組門檻 (t1, t2) 會把格子切成三段，所以每一組的平均 bits 和每一層的誤差都能用前綴和直接算出：

```
P = torch.cat([zero, H.cumsum(dim=2)], dim=2)
bits = (4.25 * N[i] + 6.25 * (N[j] - N[i]) + 8.25 * (total - N[j])) / total
err = E4[:, i] + (E6[:, j] - E6[:, i]) + (E8[:, M] - E8[:, j])   # 每一層的誤差
# 每一層的誤差不超過 NVFP4 的 10^(s/10) 倍
feasible = (i <= j) & (err <= NV[:, M] * 10 ** (slack_db / 10)).all(dim=0)
# 在可行的組合中取平均 bits 最少的；平手時選 FP4 較多的
```

論文的條件是 s = 0，也就是每一層都不比 NVFP4 差。照這個條件推導，FP6 用得太多，平均約 5.5 bits，超過作業的 5.3。所以我加了一個全域的 slack s：允許每一層的誤差比 NVFP4 多 s dB，並從 0 開始每次加 0.1 dB，取「參考模型的 effective bit rate 不超過目標」的最小 s：

```
def solve_budget(hist, target_bits, step=0.1):
    k = 0
    while True:
        slack = round(k * step, 6)
        result = solve_all(hist, slack)            # 14 組門檻
        if effective_bits(result)[2] <= target_bits:
            return result, slack
        k += 1
```

s 越大，FP4 能用的範圍越大、bits 越少，但誤差也越大，所以取最小的 s 能在 bits 上限內保留最多精度。Llama 的統計會存成 `reference_hist.pt`，換目標只需重新求解，不必再載入模型。

4. 統計實際的 bits

`run_ptq.py` 量化權重時統計每個權重 tile 的格式；評估 PPL 時，每個 Linear 的輸入 activation 進來一次就統計一次：

```
def hook(module, inp):
    a = inp[0].reshape(-1, inp[0].shape[-1])
    pad = (-a.shape[0]) % 16
    if pad:
        a = torch.cat([a, a.new_zeros(pad, a.shape[1])])
    q, fmt = adaptquant(a, t1, t2)
    counts["act"] += torch.bincount(fmt.flatten(), minlength=3)   # FP4 / FP6 / FP8 數量
    return (q[: a.shape[0] - pad].reshape(inp[0].shape),) + tuple(inp[1:])
```

每個 tile 的元素數都一樣，所以 tile 的比例就是元素的比例，平均 bits 是各格式的有效 bits 依比例加權，再加上每個 tile 的格式標記。

做法上的選擇

項目	我的做法	理由
參考模型與語料	Llama-3.1-8B、WikiText-2 train 前 2048 token	和論文相同；test split 從未使用
推導條件	每層誤差不超過 NVFP4 加 s dB，s 取最小	論文的 s = 0 會超過 5.3 bits
bits 目標	參考模型上 5.25	先試 5.15，PPL 14.93 離 15 太近，bits 還有餘裕
格式標記	每個 tile 5 bytes，算進 bits	混合精度需要記錄每個 tile 的格式，保守起見算入
權重 tile 方向	16 個輸出 row × 32 個輸入 column	讓 32 落在相乘的維度，和 MX block 一致
NVFP4 參考	16 元素 block、E4M3 scale、per-tensor FP32 scale	依 NVFP4 的公開規格
計算精度	fp16	T4 不支援 BF16 加速
Llama-3.1-8B 載入	模型放 CPU，一次搬一層到 GPU	fp16 權重約 16 GB，放不進 T4

這仍是 calibration-free 的：量化 Qwen3-4B 時沒有讓任何資料通過模型，14 個門檻都是在 Llama-3.1-8B 上算好的常數，bits 目標也只在 Llama 上檢查。

Result

最終 **PPL 為 14.64**（FP16 為 13.64），**realized effective bit rate 為 5.17 bits**，符合 ≤ 5.3 的要求。

Effective bit rate 的計算

MX 格式每 32 個元素共用一個 8-bit 的 E8M0 scale，所以每個元素的有效 bits 是：MXFP4 = 4 + 8/32 = 4.25，MXFP6 = 6 + 8/32 = 6.25。混合精度時每個 16 × 32 的 tile 還要記錄格式，5 bytes / 512 個元素 = 0.078 bits。

	MXFP4 比例	MXFP6 比例	計算	平均 bits
權重	60.22%	39.78%	0.6022 × 4.25 + 0.3978 × 6.25 + 0.078	5.1236
activation	55.21%	44.79%	0.5521 × 4.25 + 0.4479 × 6.25 + 0.078	5.2238

$$\textit{effective bit rate} = (5.1236 + 5.2238) / 2 = 5.1737 \leq 5.3$$

權重比例是 252 個 Linear 全部權重 tile 的統計；activation 比例是評估 WikiText-2 test 時，所有 Linear 輸入的所有 tile 的統計。兩者都沒有用到 MXFP8。

對照組

方法	PPL	權重 bits	activation bits	effective bit rate	論文 PPL
FP16	13.64	16	16	16	13.66
MXFP6	13.92	6.25	6.25	6.25	–
NVFP4	13.93	4.5	4.5	4.5	13.96
AdaptQuant（目標 5.25）	**14.64**	**5.12**	**5.22**	**5.17**	13.98
AdaptQuant（目標 5.15）	14.93	4.99	5.16	5.08	–
AdaptQuant（s = 0，無上限）	14.20	5.37	約 5.6（估計）	約 5.5	–
MXFP4	18.64	4.25	4.25	4.25	18.20

所有方法都同時量化權重和 activation。s = 0 的版本是加上限之前的結果，當時沒有量 activation 的 bits，依 Llama 上的比例估計約 5.6，平均約 5.5，超過上限。

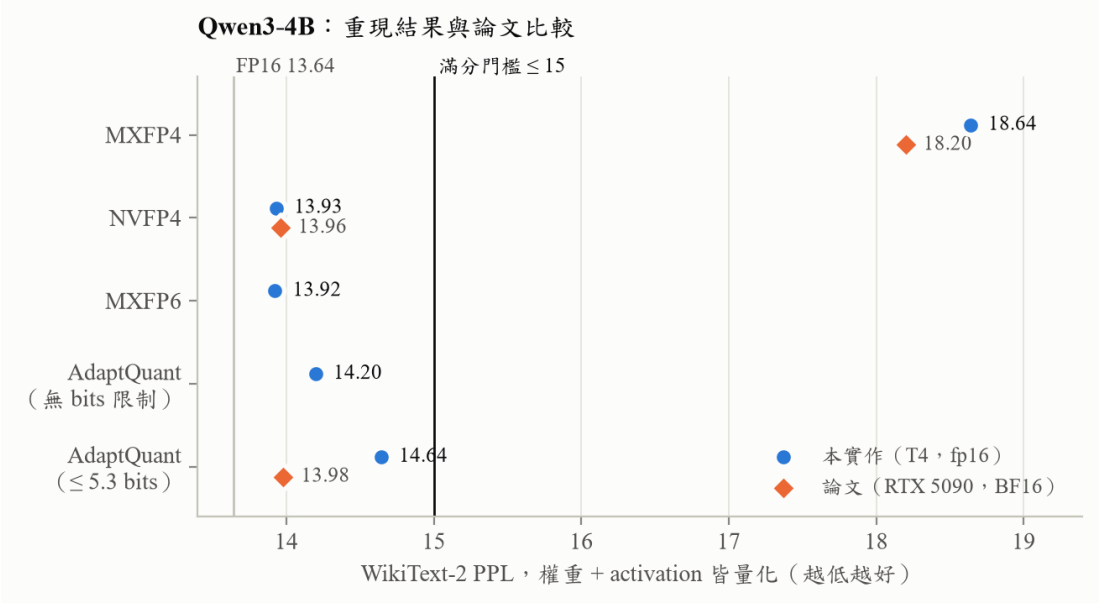


圖 1：Qwen3-4B 重現結果與論文比較

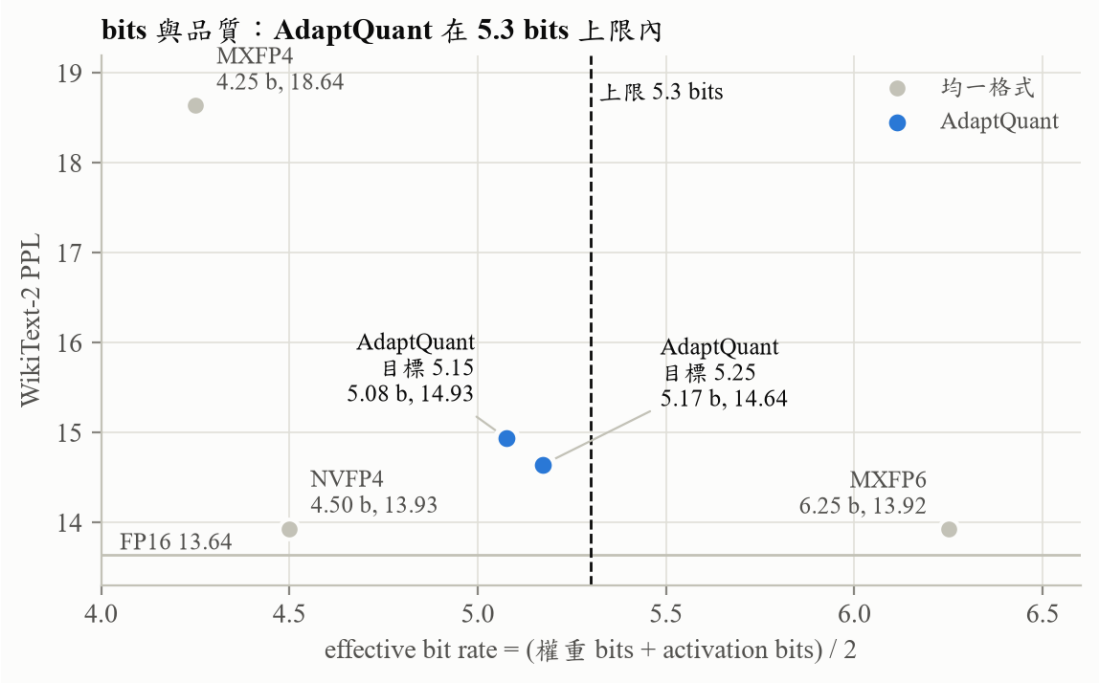


圖 2：effective bit rate 與 PPL 的關係

Observation and Discussion

推導出的門檻

最終的門檻（s = 0.7 dB）如下表與圖 3，套用到 Qwen3-4B 後權重的 stat 分布見圖 4。

projection	權重 t1	權重 FP6	act t1	act FP6
q_proj	1.90	35%	2.32	58%
k_proj	1.90	41%	2.24	63%
v_proj	1.84	33%	2.38	54%
o_proj	1.80	35%	2.40	52%
gate_proj	1.88	43%	1.88	47%
up_proj	1.80	33%	1.88	47%
down_proj	1.80	34%	2.66	58%

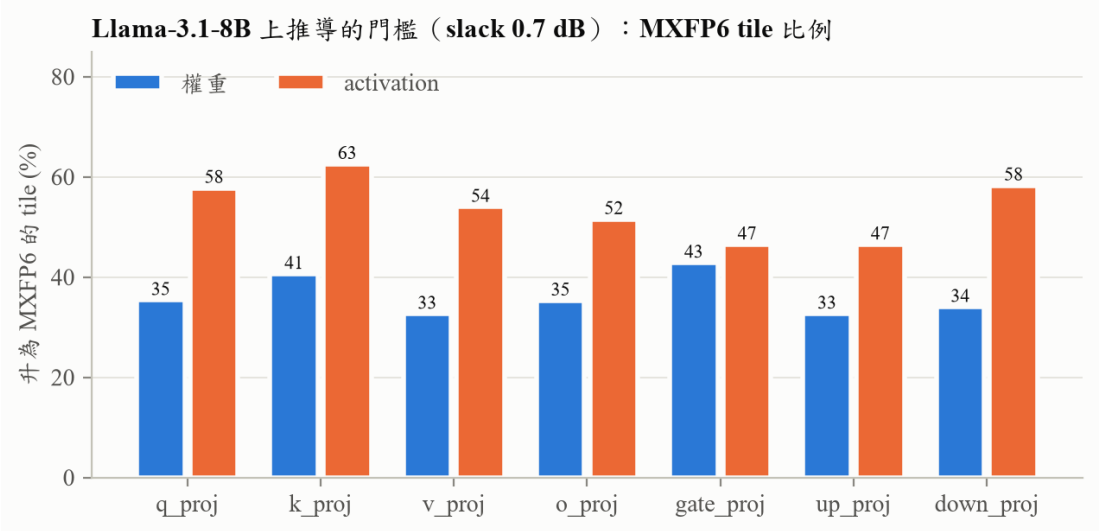


圖 3：Llama-3.1-8B 上推導的門檻對應的 MXFP6 tile 比例

和 s = 0 相比，權重的 t1 往上移了 0.06 到 0.08、activation 的 t1 往上移了 0.08 到 0.18，FP6 的比例降了 7 到 15 個百分點。所有 t2 仍然是「永遠不升級」，FP8 沒被選到，和論文一致。activation 的 FP6 比例一樣比權重高，表示 activation 比較難量化。在 Llama 上，參考模型的 effective bit rate 是 5.24；套到 Qwen3-4B 實際是 5.17，比 Llama 低，所以 5.25 的目標仍留有餘裕。

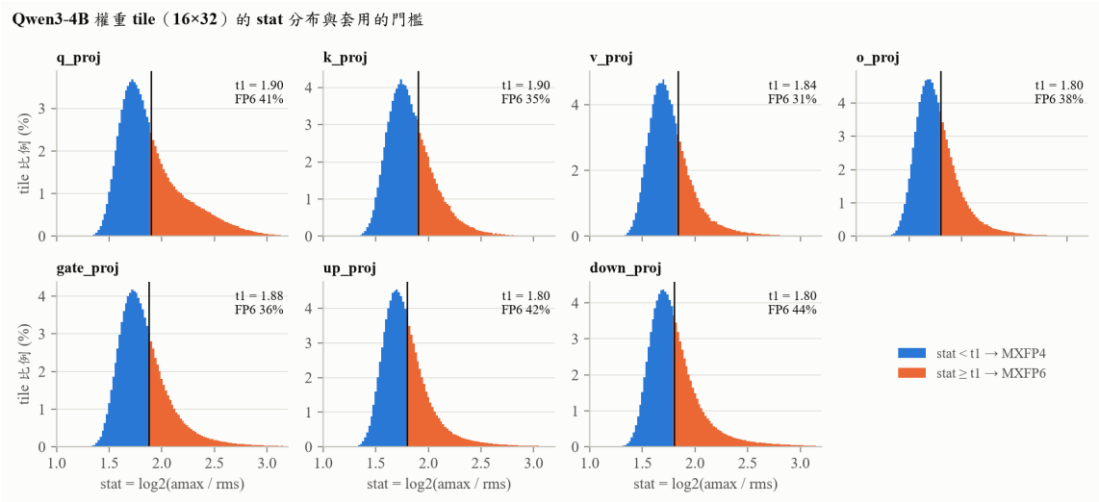


圖 4：Qwen3-4B 權重 tile（16×32）的 stat 分布與套用的門檻

對模型品質的影響

bits 和 PPL 之間是很明顯的取捨（圖 2）：effective bit rate 從 5.08 增加到 5.17，PPL 從 14.93 降到 14.64；不限制 bits 時約 5.5，PPL 是 14.20。和全部 MXFP4 相比，AdaptQuant 只多用約 1 bit，PPL 就從 18.64 降到 14.64，可見 MXFP4 的誤差主要集中在少數 stat 高的 tile，把它們改成 FP6 就能挽回大部分精度。

相對 FP16 的退化

PPL 比 FP16 高了 1.00。原因有三點：第一，為了符合 5.3 bits，權重有 60% 的 tile、activation 有 55% 的 tile 用 MXFP4，比不限制時更多。第二，activation 也被量化，而且比權重更難量化。第三，最終門檻允許每一層的誤差比 NVFP4 多 0.7 dB，所以品質會略差於 NVFP4（13.93）。

與論文差 0.66 的可能原因

1. 論文的 13.98 是 s = 0（每層不比 NVFP4 差）的結果，沒有 5.3 bits 的限制；我的版本為了符合上限放寬了 0.7 dB。不限制時我得到 14.20，和論文差 0.22。
2. 論文使用 BF16，我在 T4 上只能用 fp16。
3. 推導門檻用的 2048 個 token 不同，論文沒有說明用哪一段。
4. NVFP4 的 per-tensor scale 取法不同，也會改變推導的條件。

實作困難

最大的困難是 bits 上限。作業後來才加入這個限制，加之前我只量了權重的 bits，沒有量 activation，必須回頭加上 activation 的統計，並把 bits 目標加進門檻推導。目標要設多少也需要取捨：設太低 PPL 會接近 15，設太高又怕 Qwen 上的 bits 超過 5.3，而 Qwen 實際的 bits 要跑完才知道。

其他困難：沒有 Blackwell GPU，只能用 fake quantization，所以沒有實際加速，也沒有省到記憶體。Llama-3.1-8B 放不進 T4，推導時只好一次搬一層到 GPU。這次在伺服器上執行時硬碟非常忙，光載入模型就花了很久。為了確認量化器寫對，`test_mx.py` 檢查了元素格點、MX 的 SQNR、tile 選擇、門檻求解器和 bits 的計算；NVFP4 的 PPL 和論文只差 0.03，也可以當作交叉驗證。

限制與改進

slack 是全域的，所有 projection 放寬一樣多。不同層對誤差的敏感度不同，如果改成每種 projection 各自分配 bits，例如對 activation 較難量化的層多給一些 FP6，應該能在同樣的 5.3 bits 下得到更好的 PPL。另外，門檻是在別的模型上推導的，Qwen3 的分布不一樣，SQNR 的保證不一定能完全轉移；這次 Qwen 上的 bits 剛好比 Llama 低，但換其他模型不一定如此。